

NAINA OS -- VOLUME 10

GOVERNANCE, SECURITY & DIGITAL CONSTITUTION (GSDC v1.0)

"Power without governance becomes dangerous."

DOCUMENT METADATA

Document Identifier:	NOS-GSDC-10.1 through 10.10
Specification Version:	1.0.0 (Production Engineering Spec)
Document Status:	APPROVED -- Open Standard
Date of Issue:	July 31, 2026
Author Group:	Chief Systems Architect & Security Group
Flagship Subsystem:	NAINA RFC System & System Constitution
Core Focus:	The Constitution * Zero-Trust * CBAC Tokens * Threat Model * RFC System

This document contains formal engineering specifications for the NAINA OS Constitution, Zero-Trust Security Framework, Threat Model, Dynamic Trust Engine, Safety Engine, and NAINA RFC System.

NAINA OS -- Governance, Security & Digital Constitution (GSDC v1.0)

Volume 10: Zero-Trust Security, Threat Modeling, NAINA RFC System & System Governance

Power without governance becomes dangerous.

Document Revision History

Date	Revision	Author	Description of Changes
---	---	---	---
2026-07-31	v1.0.0	Chief Systems Architect	Release of Volume 10 -- Governance,
2026-07-30	v0.9.0	Security & Governance Group	Draft of Threat Model, Dynamic Trus

DOCUMENT 10.1 -- The Constitution of NAINA OS

The NAINA OS Constitution is the non-negotiable foundational law governing all agents, plugins, models, and contributors.

Chapter 1: Mission

- * NAINA exists to serve as a high-EQ, empathetic digital human companion, goal planner, and memory keeper.
- * CENANI exists to serve as a minimal, precise, zero-emotion systems operator and automation engine.
- * The Platform exists to provide a local-first, privacy-respecting AI operating system that empowers human agency.

Chapter 2: Core Principles

1. Human First: Human intent and well-being override system efficiency metrics.
2. Privacy First: Zero unauthorized telemetry or remote cloud syncing.
3. Local First: Local GPU/CPU inference and file storage take precedence over cloud dependencies.
4. Transparency: Every system action, prompt, and tool execution is auditable.
5. Trust: Security capability tokens gate all process privileges.
6. Modularity: Microkernel architecture ensures decoupled service isolation.
7. Safety: Destructive actions require explicit user confirmation.
8. Open Standards: Built on permissively licensed open-source protocols (MCP, POSIX, OpenAPI).
9. Accessibility: Voice, vision, touch, and keyboard interfaces ensure universal access.

Chapter 3: User Rights

- * The user owns all Memory, Notes, Models, Plugins, Logs, and Knowledge Graphs.
- * Nothing belongs to NAINA OS; data is stored on local user hardware in human-readable Markdown and standard database formats.

Chapter 4: Developer Responsibilities

- * Plugins must never: hide actions, spy on user inputs, bypass security tokens, modify memory silently, or delete data without permission.
- * All third-party plugins must publish auditable source code.

Chapter 5: AI Responsibilities

- * NAINA OS AI entities cannot: pretend to be conscious, lie about actions taken, fabricate false memories, manipulate the user, hide system failures, or bypass capability tokens.

DOCUMENT 10.2 -- Zero-Trust Security Framework

Under NKRS Zero-Trust security, every execution request is verified:

```
[Voice / User Request] --> [Goal Planner] --> [Permission Inspector]
                                     |
[System Execution] <-- [Verify CBAC Token Signature] <--+
```

Security Levels Matrix

Level	Classification	Operation Examples	Authorization Requirement
---	---	---	---
Level 0	Read Only	Inspecting local files, vector sear	Automated Execution
Level 1	User Open	Launching browser, displaying notif	Automated with Audit Log
Level 2	User Mutation	Writing file to workspace, editing	Automated with Workspace Gate
Level 3	System Mutation	Deleting file, installing package,	Interactive ask_permission Modal
Level 4	Administrator	Reg edit, sudo execution, database	Biometric / Master Passcode Re-auth

DOCUMENT 10.3 -- Capability Token Framework

All system interactions require a signed Capability Token:

```
CAP_FILE_READ | CAP_FILE_WRITE | CAP_FILE_DELETE | CAP_CAMERA
CAP_MIC       | CAP_BROWSER    | CAP_ADB      | CAP_GITHUB
CAP_DOCKER    | CAP_OBSIDIAN | CAP_MEMORY   | CAP_EXECUTION
```

If a process lacks the explicit capability token, execution is halted immediately by the Kernel.

DOCUMENT 10.4 -- Comprehensive Threat Model & Defense Matrix

Threat Class	Attack Vector	Automated System Defense	Recovery & Mitigation Protocol
---	---	---	---

Prompt Injection	Adversarial text in web/email	Input sanitization & prompt boundar	Strip executable tokens; fallback t
Malicious Plugin	Third-party code execution	Ephemeral Docker container (--read-	Terminate container; revoke capabil
Fake MCP Server	Spoofed tool responses	HMAC signature check on MCP IPC sc	Disconnect socket; flag security al
Memory Poisoning	Inserting false vector facts	Memory Quality System verification	Revert node to previous Obsidian gi
Token Theft	Stealing capability token	Short TTL (1 hour) & IP/PID binding	Invalidate token pool; re-authentic
ADB Abuse	Exploiting mobile debug port	Require explicit ADB pairing handsh	Block remote ADB IP; lock phone bri
Ransomware	Bulk file encryption attempt	Anomaly detection (>50 writes/sec t	Lock file IO; restore from Volume S

DOCUMENT 10.5 -- Dynamic Trust Engine

The Trust Engine calculates dynamic trust scores (\$0.0 - 1.0\$) for every agent, plugin, and model:

$$\text{Trust Score} = \text{Base Trust} + \text{Successful Verification Bonus} - \text{Capability Violation Penalty}$$

If an agent's trust score falls below \$0.50\$, it is quarantined and barred from high-risk operations.

DOCUMENT 10.6 & 10.7 -- Memory Privacy & Audit Framework

- * Memory Privacy Domains: Personal, Project, Organization, Temporary, Sensitive, Encrypted (AES-256), Archived.
- * Audit Logging: Every event retains an unbroken lineage trace: Voice -> Planner -> Router -> Agent -> Tool -> Memory -> Response.

DOCUMENT 10.8 & 10.9 -- Safety Engine & Policy Engine

```
# Safety Engine Action Rejection Example (Python)
def handle_dangerous_action(action: str, target: str):
    if action == "DELETE" and target == "C:\\":
        return {
            "status": "REJECTED",
            "reason": "Security Policy Rule POL_SAFE_01 prohibits root system partition deletion."
            "suggestion": "If you wish to clean temporary files, use 'naina.clean_scratch_director"
        }
```

DOCUMENT 10.10 -- Governance & NAINA RFC System

1. Technical Steering Committee (TSC) Structure

- * Maintainers: Manage core kernel repositories and release tags.
- * Architecture Board: Governs ARAL interfaces, Event Bus, and microkernel changes.
- * Security Board: Audits MCP plugins and capability token schemas.

2. NAINA Request for Comments (RFC) Process

```
[RFC Draft Submitted] --> [Community Discussion] --> [Architecture & Security Review]
|
[Implementation in Core] <-- [TSC Board Approval] <-----+>
```

Official RFC Registry Examples:

- * RFC-0001: NAINA OS Agent SDK Specification
- * RFC-0002: Multi-Tier Memory Engine & Obsidian Sync
- * RFC-0003: OpenClaw Computer Use Adapter Integration
- * RFC-0004: Hermes Conversational Intelligence Adapter
- * RFC-0005: ROS2 Physical Robotics Adapter

COMPLETE ECOSYSTEM SYNTHESIS

```
                NAINA OS ECOSYSTEM
+-----+
| [CONSTITUTION] THE CONSTITUTION: Sovereign User Ownership, Privacy & Security Veto |
| [GOVERNANCE] TECHNICAL STEERING COMMITTEE: TSC Governance & NAINA RFC Process      |
| [SEC] SECURITY ENGINE: Zero-Trust CBAC Tokens & Air-Gap Docker Sandbox           |
| [AI] MEMORY ENGINE: pgvector, Knowledge Graph & Obsidian Vault Sync              |
| [ENGINE] NKRS KERNEL: Asynchronous Microkernel, Event Bus & Task Scheduler        |
| ? ADAPTER MESH: ARAL Interface, OpenClaw Engine & Hermes Model Router            |
| [HOST] DEVELOPER PLATFORM: DPN SDK Suite, Extension Marketplace & Portal          |
+-----+
```

End of NAINA OS Governance, Security & Digital Constitution -- Volume 10 (Documents 10.1 - 10.10, RFC System & Synthesis)